实时数仓：热点、窗口与UV

从一次新闻点击，到任意时间窗口的去重人数；拆解企业实时数仓的存储、检索与指标聚合三层原理。

通用数仓原理　本地PG链路对照　代码锚点已核对

12KB
HLL固定空间
1万到10亿UV，状态大小不变

0.8%
HLL标准误差
16384桶；1.04/√m，与基数无关

[s,e)
半开时间区间
相邻窗口不重不漏的统一口径

三层技术栈，本册聚焦第三层

存储层 · 数据怎么存
列存Parquet/ORC、分区、编码压缩、排序键、LSM/MergeTree后台合并。

检索层 · 查询怎么快
MPP并行执行、向量化计算、谓词下推、分区裁剪、物化视图自动路由。

指标聚合层 · 指标怎么算
预聚合、固定/滑动窗口、HLL与Bitmap可合并状态、去重语义——本册主线。

这本图解的阅读顺序

第2-3页：事件流与时间窗口；第4-6页：UV三方案、HLL与Bitmap底层；第7页：列存格式与窗口检索；第8页：一次点击的字段演化与本地PG对照。

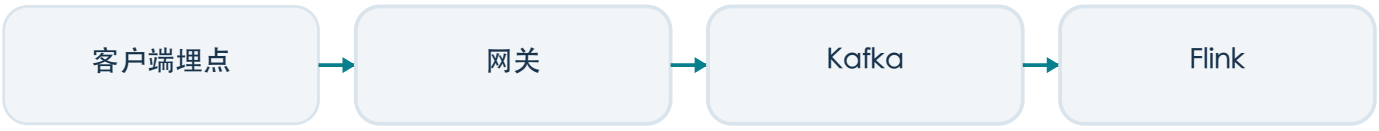
本册讲解通用企业数仓原理（Doris/ClickHouse/Redis一类），第8页才对照本项目本地合成链路；不构成企业接口或数仓的验收结论。

一次点击，先变成事件流

不存在“news_id的访问量字段”这种底层存储；每次点击先落成一条不可变事件，计数只是它的衍生视图。

埋点事件 JSON（最原始的事实）

```
{
 "event_id":  "e9f3a1c2-7d20-...",  // 幂等：重试不重复计数
 "event_type": "click",
 "news_id":   "N123",
 "tenant_id":  10001,
 "user_id":   "u10086",
 "session_id": "s7b2e90...",
 "event_time": 1770000123456,  // 客户端毫秒时间戳
 "device":    {"os":"iOS","model":"iPhone15"}
}
```



Kafka分区键 = user_id哈希；事件可重放，是真正的事实源

A · Redis 在线层

每次点击：ZINCRBY hot:rank:2026101210 1 "N123"；PFADD hot:uv:2026101210 "u10086"。窗口 = key命名，生命周期 = TTL。

B · OLAP 分析层

Flink按(租户,新闻,小时桶)聚合成增量行写入聚合表；存储引擎后台merge按声明聚合，写入方不关心重复。

核心原则：明细日志是 source of truth

计数器是可丢弃的缓存。只存计数，改窗口口径、补UV、按品类重切分都无从重算；存了明细，一切口径都能回放重建。

本地项目没有这条事件流：合成数据直接生成小时桶指标，详见第8页。

窗口档位与两种维护方式

窗口大小没有标准值，档位由指标类型决定；窗口在Redis和数仓里的“本体”完全不同。

场景	典型窗口	说明
监控告警	10s / 1min	计数类速率指标，不做UV去重
实时热榜	滚动5/10min，或近1h滑动	滑动窗口是主流：粒度保鲜度，跨度保样本量
小时桶	滚动1h	精确UV的最小实用粒度；与同小时基线对齐
天级榜单	自然日 + T+1回刷	实时天窗先行，离线修正迟到数据

窗口的两种本体

Redis：窗口 = key命名

10点的点击只写 ...2026101210 这个key。推进 = TTL到期整key删除；固定窗口直读一个key；任意窗口只能合并预定义桶。

数仓：窗口 = event_time列

窗口根本不需要维护。查询WHERE event_time过滤，天然任意窗口；时间推进由分区滚动归档管理。

选型逻辑

01 窗口越小UV越不可信：分钟级只做计数或HLL近似，精确UV从1h起步。

02 滑动窗口避免突变：“近1h每1min滑”防止整点切换时榜单跳变。

03 窗口≠数据就绪：水位watermark加迟到容忍5-15min，完整窗口还要再等几分钟。

本地链路固定单小时整点桶；三重校验见第8页。

UV = 集合基数，三种实现

去重是集合语义不是数值语义：9-10点UV=100，10-11点UV=100，两桶可能含同一批用户，相加必重复计数。

HashSet

精确。存全量ID，O(n)空间；合并要求交换全部明细。适用调试与小数据。

Bitmap

精确。一位一用户，popcount即UV；支持交并差。需全局编号，稀疏必须压缩。

HLL

近似约0.8%。状态固定12KB；只对并集封闭；合并=逐桶max，代价极小。实时链路主力。

跨窗口合并：求并集，不是相加

方案	合并方式	结果
HashSet	两集合求并集	精确，但要交换全量明细
Bitmap	逐位 OR	精确，代价小
HLL	逐桶取 max	近似（误差1%量级），代价极小

任意窗口UV的根基：可合并性

小时桶各存一份可合并状态（Doris HLL列、ClickHouse uniqCombinedState），查询时窗口内的桶现场merge，得到该窗口去重人数。

HLL只对并集封闭

交集没有直接算法，只能容斥：$E(A \cap B)=E(A)+E(B)-E(A \cup B)$，误差被放大；需要交集/差集分析时改用Bitmap。

HLL底层：从抛硬币到12KB

hash把任意ID变成均匀随机比特串，去重问题就变成抛硬币问题；剩下的工作都是压方差。

01

hash均匀化

同一ID永远同一hash（所以能去重）；不同窗口对同一桶取max可合并（所以能并集）。hash质量差直接带来系统偏差。

02

前导零 = 抛硬币

连续k个正面平均需2^(k+1)次抛掷；观察到最大前导零K，则UV约等于2^K。单估计器方差O(n²)，一次好运的hash就能让估计翻倍。

03

分桶随机平均

hash高b位选桶（b=14即m=16384），每个桶独立估计再平均；标准误差降为1.04/√m，且与基数n无关。

04

调和平均 + 两端修正

E = α_m · m² / Σ2^(-M[j])。小基数切LinearCounting（空桶数Z：m · ln(m/Z)）；64bit hash空间大，实际永不饱和。

16384桶 × 6bit

= 12KB固定状态

误差 ≈ 0.81%

1.04/√16384，与UV规模无关

merge = 逐桶max

满足交换律与结合律

合并的正确性

并集流的每桶最大前导零 = max(两份对应桶)。满足交换与结合律：任意小时桶拼成的大窗口，与在整窗口上直算一遍，分布完全相同。

精确一侧：Roaring Bitmap

32bit空间切成2^16个高16位块；每块按密度三选一容器，空间与速度兼得。

容器	结构	使用条件
ArrayContainer	有序uint16数组	元素 < 4096（稀疏）
BitmapContainer	2^16bit = 8KB定长位图	⩾ 4096（稠密）
RunContainer	游程 (start,length) 对	连续区间，如1~9999

4096这个阈值不是拍脑袋

数组每个元素2字节，k个元素占2k字节；位图恒定8KB。2k = 8192B 即 k = 4096 时两边打平——按空间自动选容器。

集合操作

UV = 各容器popcount求和（CPU的POPCNT硬件指令，一周期处理64bit）。并集 = 逐容器OR，只增稠可升级容器；交集 = AND，变稀可降级。

精确 vs 近似

方案	状态	误差	适用
HashSet	O(n)全量ID	精确	调试与小数据
Roaring Bitmap	压缩位图	精确	对账、画像圈选、交并差
HLL	12KB固定	约0.8%	分钟级实时与任意窗口合并

列存格式与窗口检索

数据按列重组、按时间有序；一条时间范围查询经过四层过滤，只付窗口内数据的成本。

物理布局（按列重组，同列聚集）

news_id　[N123][N124][N123][N125] → 字典编码成 [1][2][1][3]

event_time [10:00][10:00][10:01][10:02] → 有序；delta+RLE压缩比极高

clicks　[1][1][1][1][1] → 大段重复；RLE一行搞定

排序键

(event_time, news_id)物理有序；时间有序是窗口检索快的第一功臣。

分区

按天/月目录级切分；旧分区整目录落冷盘或删除。

分桶

hash(news_id)水平拆到多机；支撑MPP并行扫描。

聚合模型：声明合并语义，不更新只追加

```
CREATE TABLE hot_hourly (
  tenant_id BIGINT, news_id VARCHAR, event_time DATETIME,
  clicks BIGINT SUM AGGREGATE,    -- 同key重复导入自动SUM
  uv HLL HLL_UNION AGGREGATE      -- 同key自动合并HLL状态
) AGGREGATE KEY(tenant_id, news_id, event_time);
```

ClickHouse对应AggregatingMergeTree与uniqMerge；本地PG没有这一层，合成数据一次生成。

窗口检索的四层过滤

01	**分区裁剪**	时间条件只打开相关分区目录，其余全部跳过。
02	**ZoneMap / 稀疏索引**	每个数据块记录event_time的min/max，几十字节跳过一个块。
03	**排序键定位**	news_id前缀有序，二分定位到连续块。
04	**向量化扫描**	剩余块按列批量加载，SIMD聚合。

写入走LSM：memtable+WAL → 刷小文件 → 后台merge按声明聚合；“水位”即压实进度——窗口完整性的来源。

字段演化与本地PG对照

同一次点击在企业链路与本地链路的落点；本地是“schema语义相同、省掉引擎”的教学实现。

u10086 点击 N123（10:23:05）之后的字段演化

↓ 埋点JSON：10+字段，user_id + 毫秒event_time

↓ Kafka：事件原样入分区，key = user_id哈希

↓ Flink消费，同一份流分两路

A → Redis：ZINCRBY +1 / PFADD 入桶HLL（明细身份仍在）

B → 小时桶增量行 → OLAP聚合引擎merge成一行

计数列SUM；UV列 = 可合并HLL状态（约12KB）

↓ 本地PG：unique_users = 精确bigint（8字节，不可合并）

本地链路：dw schema 三表一视图

对象	内容	位置
dw.dim_news	新闻维表：title/category/publish_time	warehouse.py
dw.news_metric_hourly	小时桶指标：clicks/unique_users/时长/互动	warehouse.py
dw.news_metric_baseline_hourly	同小时基线参考均值	warehouse.py
dw.news_behavior_aggregate	18列冻结视图；契约SHA256校验	warehouse.py:106

索引 (tenant_id, event_time, news_id) 见 warehouse.py:447；查询统一半开区间 event_time >= :start AND < :end。

窗口三重校验（native_hot_news_sql_support.py:113）

恰好1小时 + 整点对齐 + 落在冻结数据集覆盖内；跨小时请求在对话层直接拒绝（query_understanding.py:430，window_not_approved），因为bigint UV无可合并状态。

生成链：代码锚点核对 verify_sources() → WarehouseUvPrinciplesGuide / ReportLab → 嵌入中文字体的8页PDF。参数：--output / --font / --bold-font。

TODO：企业数仓Adapter、真实跨小时去重与调度语义未验收；本册数字为原理示例，非运行数据。